

РЕФЕРАТ

Расчетно-пояснительная записка 38 с., 8 рис., 1 табл., 6 источн., 1 прил.

Ключевые слова: TCP, HTTP, статический сервер, нагрузочное тестирование.

Цель работы — разработка сервера для отдачи статического содержимого с диска по протоколу HTTP.

В ходе выполнения работы были описаны основные компоненты веб-сервера и принципы его работы, описан алгоритм работы статического сервера, реализован статический сервер и проведен нагрузочное тестирование разработанного программного обеспечения.

По результатам исследования был сделан вывод о том, что оптимальная конфигурация по числу потоков слушателей и потоков обработчиков равна (2, 8).

СОДЕРЖАНИЕ

РЕФЕРАТ	3
ВВЕДЕНИЕ	6
1 Аналитический раздел	7
1.1 Анализ сокетов	7
1.2 Анализ основных протоколов	8
1.3 Анализ системных вызовов	9
2 Конструкторский раздел	13
2.1 Алгоритм работы главного потока сервера	13
2.2 Алгоритм работы сервера	14
2.3 Алгоритм работы потока слушателя	15
2.4 Алгоритма работы потока обработчика	16
3 Технологический раздел	18
3.1 Средства реализации	18
3.2 Реализация алгоритма работы потока слушателя	18
3.3 Реализация алгоритма работы потока обработчика	22
3.4 Реализация алгоритма работы главного потока сервера	30
4 Исследовательский раздел	32
4.1 Условия проведения исследования	32
4.2 Результаты тестирования	33
4.3 Исследование влияния количества потоков на пропускную спо- собность	34
4.4 Исследование влияния количества потоков на задержку	34
4.5 Исследование влияния количества потоков на скорость передачи	34
4.6 Исследование конфигурации $L=2, W=8$	35
4.7 Выводы	36
ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	38

ВВЕДЕНИЕ

Основной моделью взаимодействия между двумя и более конечными устройствами в компьютерной сети является модель клиент-сервер. Под термином компьютерная сеть будем понимать набор автономных компьютеров, связанных одной технологией. Причём назовём два компьютера связанными, если они могут обмениваться информацией.[1] Таким образом, приложение, реализующее данную модель взаимодействия, имеет два типа процессов:

- клиент - это процесс, который обращается к серверу за ресурсом;
- сервер - это процесс, который обладает ресурсом и обслуживает клиентов.

Для обмена информацией приложение должно использовать протоколы прикладного уровня, в частности HTTP. В терминах приложения веб-сервер - это процесс сервер, который прослушивает определенный порт для обслуживания клиента, реализует взаимодействие с клиентом посредством протокола HTTP, и распоряжается, имеющимися ресурсами.

Цель работы - разработка сервера для отдачи статического содержимого с диска по протоколу HTTP.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать архитектуру компьютерной сети, протоколы и системные вызовы;
- описать основные компоненты и алгоритмы работы веб-сервера.
- реализовать сервер;
- провести нагрузочное тестирование разработанного сервера;

1 Аналитический раздел

Архитектура компьютерной сети - это набор уровней и протоколов, определяющих принципы построения и функционирования сети. Она позволяет различному аппаратному и программному обеспечению взаимодействовать друг с другом.[2] Основными принципами, заложенными в архитектуру компьютерной сети, являются:

- принцип абстракции - любой уровень должен предоставлять сервисы вышестоящим уровням, скрывая от них детали реализации;
- принцип взаимодействия объектов - взаимодействие объектов, находящихся на одном уровне, производится посредством протоколов;
- принцип интерфейса - взаимодействие между соседними уровнями реализуется с помощью интерфейсов.

Одной из моделей архитектуры компьютерной сети является модель TCP/IP. Она регламентирует необходимые уровни и протоколы. В рамках модели TCP/IP, сокет определяется как конечная точка взаимодействия, и в тоже время сокет является интерфейсом, который создает уровень абстракции между приложением и сетью. Он позволяет процессам использовать стандартные вызовы для обмена данными, скрывая детали реализации архитектуры компьютерной сети. Таким образом, сокет является элементом архитектуры компьютерной сети.

1.1 Анализ сокетов

В операционной системе Linux сокет представлен в виде файлового дескриптора. Для создания используется системный вызов `socket()`, который принимает три параметра: домен, тип и протокол.[3] Домен определяет пространство адресов, тип задаёт семантику передачи данных, а протокол конкретизирует используемое множество сетевых протоколов.

Основными доменами для взаимодействия по сети являются `AF_INET` для поддержки пространства адресов IPv4 и `AF_INET6` для поддержки пространства адресов IPv6.

Основным типом для передачи данных является `SOCK_STREAM`, который обеспечивает последовательную, надежную, двустороннюю связь на ос-

нове TCP. и SOCK_DGRAM, который обеспечивает ненадежную передачу сообщений фиксированной длины на основе UDP.

Стоит отметить, что важным свойством сокета является дуплексная связь - это означает, что и процесс сервер, и процесс клиент могут использовать, сокет как для получения данных, так и для их отправки.

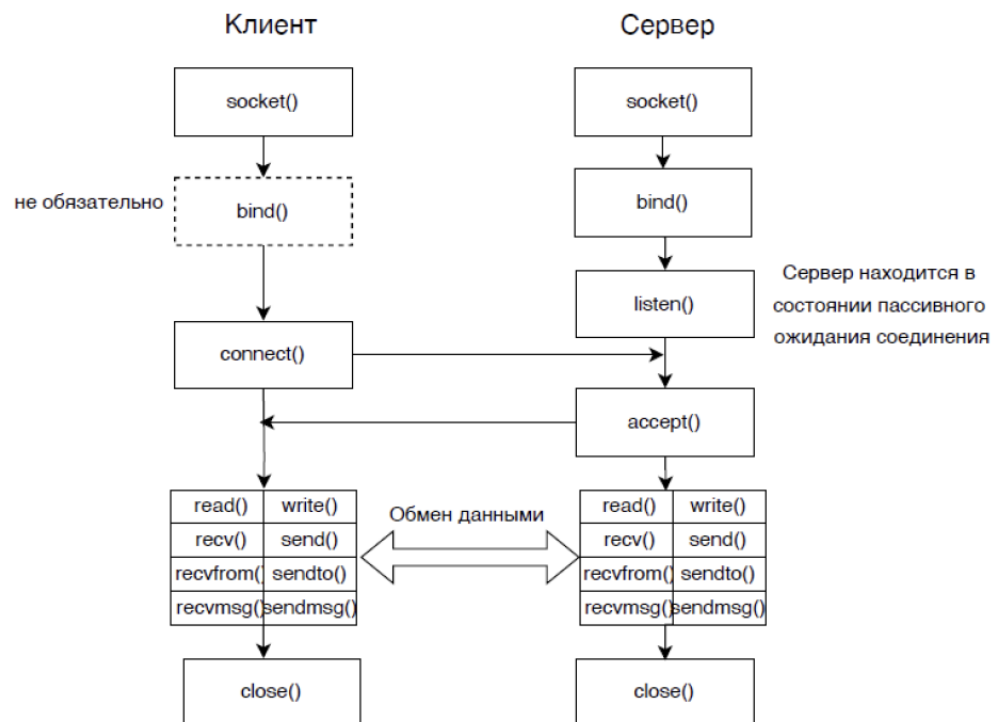


Рисунок 1.1 – Схема взаимодействия на сокетах

1.2 Анализ основных протоколов

Протокол - это правила и соглашения между сущностями одного уровня архитектуры компьютерной сети, которые определяют формат передачи данных и процедуры, необходимые для передачи информации.[1]

На транспортном уровне модели TCP/IP выделяют два основных протокола TCP и UDP. TCP (Transmission Control Protocol) - это протокол с установлением соединения, обеспечивающий надежный сквозной байтовый поток по ненадежной интернету. Здесь надежность означает то, что протокол гарантирует доставку данных без ошибок, потерь и дублирования. А установление соединения означает то, что, перед обменом информацией, необходимо совершить «трехстороннее рукопожатие»

UDP (User Datagram Protocol) - это ненадежный протокол без установле-

ния соединения.

Протокол прикладного уровня HTTP (HyperText Transfer Protocol), используемый для передачи веб-страниц и в большинстве случаев работающий поверх протокола TCP, работает по модели запрос-ответ.

Модель запрос-ответ регламентирует взаимодействие между процессом клиентом и процессом сервером. Так, основными методами процесса клиента по протоколу HTTP являются:

- GET - получение
- POST - добавление
- DELETE - удаление
- PATCH - неполное изменение
- PUT - полное идемпотентное изменение

А основные состояния, которые процесс сервер возвращает в виде кодов, согласно протоколу HTTP, являются:

- 1xx - информационные
- 2xx - успех
- 3xx - перенаправление
- 4xx - ошибка клиента
- 5xx - ошибка сервера

1.3 Анализ системных вызовов

Помимо предоставления абстракции конечной точки взаимодействия - сокета, операционная система Linux поддерживает механизм потоков и мультиплексирования.

1.3.1 Анализ потоков в Linux

Выделяют следующие типы потоков:

- потоки ядра (kernel threads) - создаются внутри ядра с помощью `kthread_create()`. Не имеют пользовательского адресного пространства (`mm = NULL`), выполняются исключительно в пространстве ядра, которое разделяют со всеми остальными потоками.
- пользовательские потоки в модели M:1 - создаются и планируются библиотекой в пользовательском пространстве без участия ядра. Ядро видит только один поток, а библиотека-менеджер самостоятельно распределяет процессорное время. Недостаток - блокирующий системный вызов в одном потоке блокирует все потоки процесса, так как ядро не различает их; [4]
- пользовательские потоки в модели 1:1 - каждому пользовательскому потоку соответствует ровно один поток ядра. В Linux стандартная библиотека `glibc` реализует данную модель через подсистему NPTL (Native POSIX Threads Library): вызов `pthread_create()`[3] порождает поток ядра через системный вызов `clone()` с флагами `CLONE_VM`, `CLONE_FILES`, `CLONE_THREAD`. [5] Планирование полностью делегируется планировщику ядра (CFS), что устраняет проблему блокировки модели M:1.

Основным свойством потоков является наследование адресного пространства родителя: для пользовательских потоков - это адресное пространство процесса родителя, а для потоков ядра - пространство ядра, общее для всех потоков. Из данного свойства следует необходимость обеспечения синхронизации доступа к разделяемым ресурсам с помощью примитивов, таких как мьютексы или семафоры.

1.3.2 Анализ мультиплексирования ввода-вывода в Linux

Мультиплексирование ввода-вывода - это механизм, позволяющий одному потоку выполнения одновременно отслеживать состояние готовности нескольких файловых дескрипторов. В контексте веб-сервера это означает возможность обслуживать множество клиентских соединений без выделения отдельного потока на каждое из них.

В ОС Linux реализованы следующие системные вызовы мультиплексирования:

- `select()` - принимает три множества дескрипторов (чтение, запись, исключения), представленных битовыми масками `fd_set`. Ограничение - максимальный номер дескриптора определяется константой `FD_SETSIZE` (по умолчанию 1024). При каждом вызове ядро линейно проверяет все дескрипторы в множестве, что даёт сложность $O(n)$, где n - максимальный номер дескриптора; [6]
- `poll()` - принимает массив структур `pollfd`, каждая из которых содержит дескриптор, запрашиваемые и возвращённые события. Снимает ограничение `FD_SETSIZE`, однако сложность остаётся $O(n)$, так как ядро проверяет весь массив;
- `epoll` - подсистема, состоящая из трёх системных вызовов: `epoll_create()` создаёт экземпляр `epoll`, `epoll_ctl()` регистрирует или удаляет дескрипторы, `epoll_wait()` блокирует поток до наступления события. В отличие от `select()` и `poll()`, ядро поддерживает внутреннюю структуру данных и возвращает только готовые дескрипторы, что даёт сложность $O(k)$, где k - количество готовых дескрипторов. [6]

При регистрации дескриптора через `epoll_ctl()` указываются флаги событий, которые необходимо отслеживать:

- `EPOLLIN` - дескриптор готов к чтению;
- `EPOLLOUT` - дескриптор готов к записи;
- `EPOLLERR` - произошла ошибка на дескрипторе;
- `EPOLLHUP` - разрыв соединения;
- `EPOLLRDHUP` - удалённая сторона закрыла соединение или запись;
- `EPOLLONESHOT` - после срабатывания дескриптор деактивируется до повторной регистрации через `epoll_ctl()`.

`epoll` поддерживает два режима срабатывания:

- `level-triggered` (по умолчанию) - событие генерируется, пока дескриптор остаётся в состоянии готовности;

— edge-triggered (флаг - EPOLLET) - событие генерируется однократно при переходе дескриптора в состояние готовности.

Таким образом, для реализации многопоточного веб-сервера epoll является предпочтительным механизмом мультиплексирования, поскольку его производительность в теории не деградирует с ростом числа одновременных соединений.

2 Конструкторский раздел

В данном разделе описаны алгоритмы работы сервера: алгоритм главного потока, алгоритм общей схемы работы, алгоритм потока слушателя и алгоритм потока обработчика.

2.1 Алгоритм работы главного потока сервера

На рисунке 2.1 представлена схема алгоритма работы главного потока сервера.

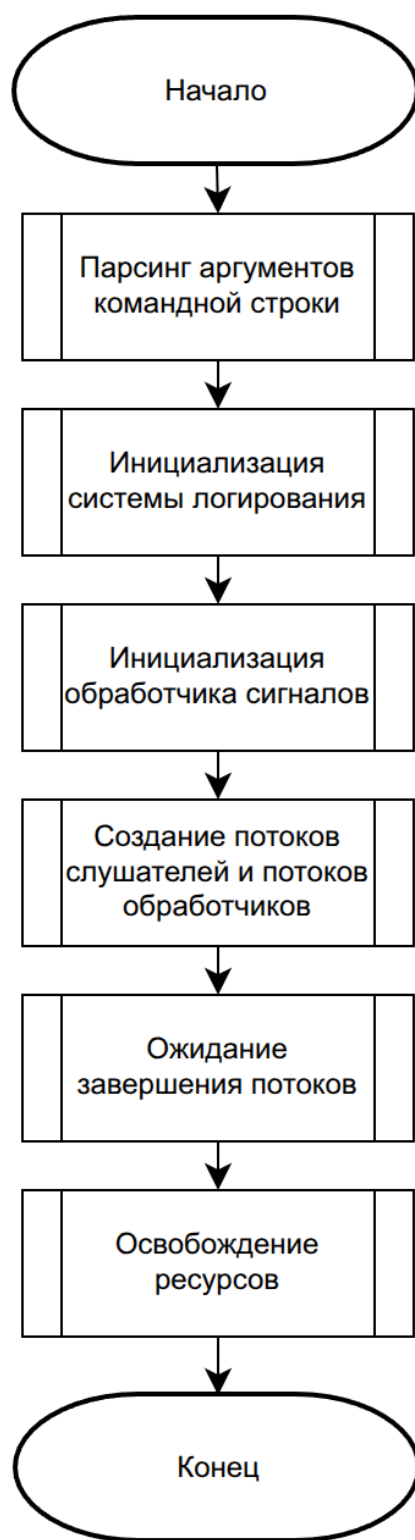


Рисунок 2.1 – Схема алгоритма работы главного потока сервера

2.2 Алгоритм работы сервера

На рисунке 2.2 представлена схема алгоритма работы сервера.

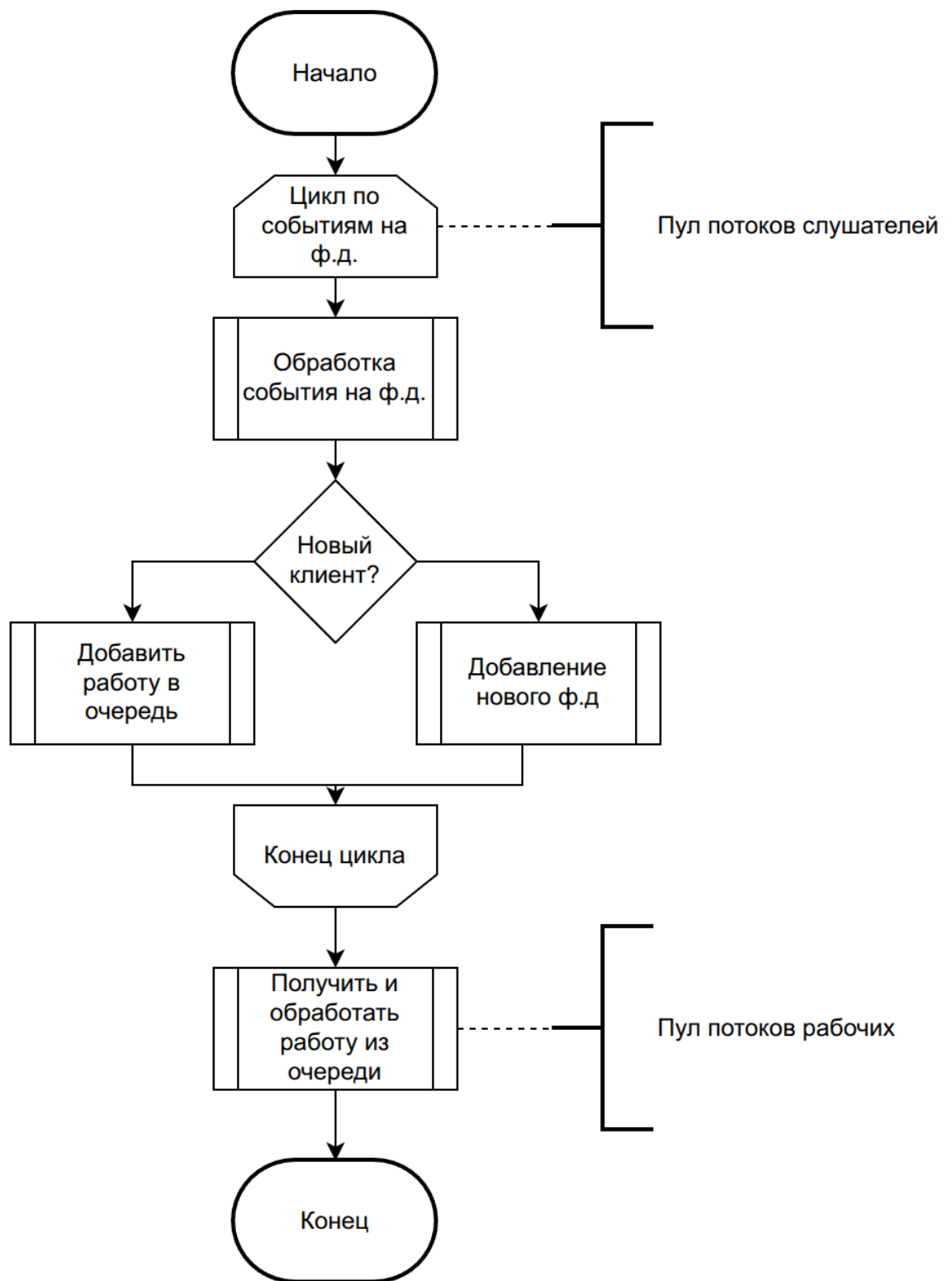


Рисунок 2.2 – Схема алгоритма работы сервера

2.3 Алгоритм работы потока слушателя

На рисунке 2.3 представлена схема алгоритма работы потока слушателя.

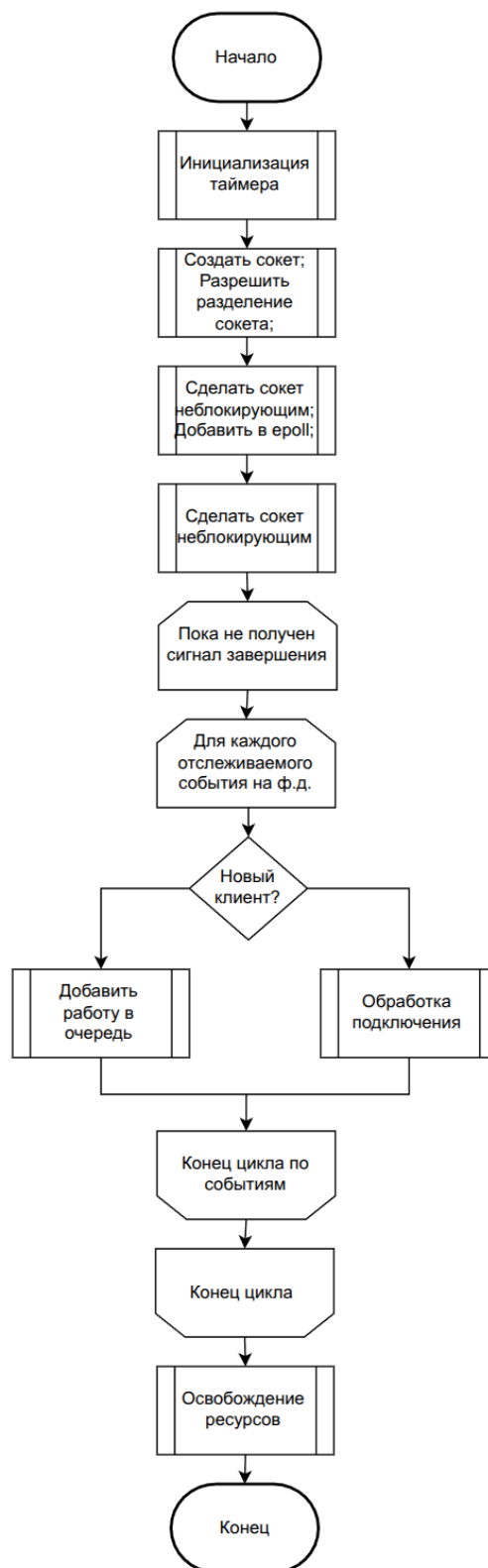


Рисунок 2.3 – Схема алгоритма работы потока слушателя

2.4 Алгоритма работы потока обработчика

На рисунке 2.4 представлена схема алгоритма работы потока обработчика.

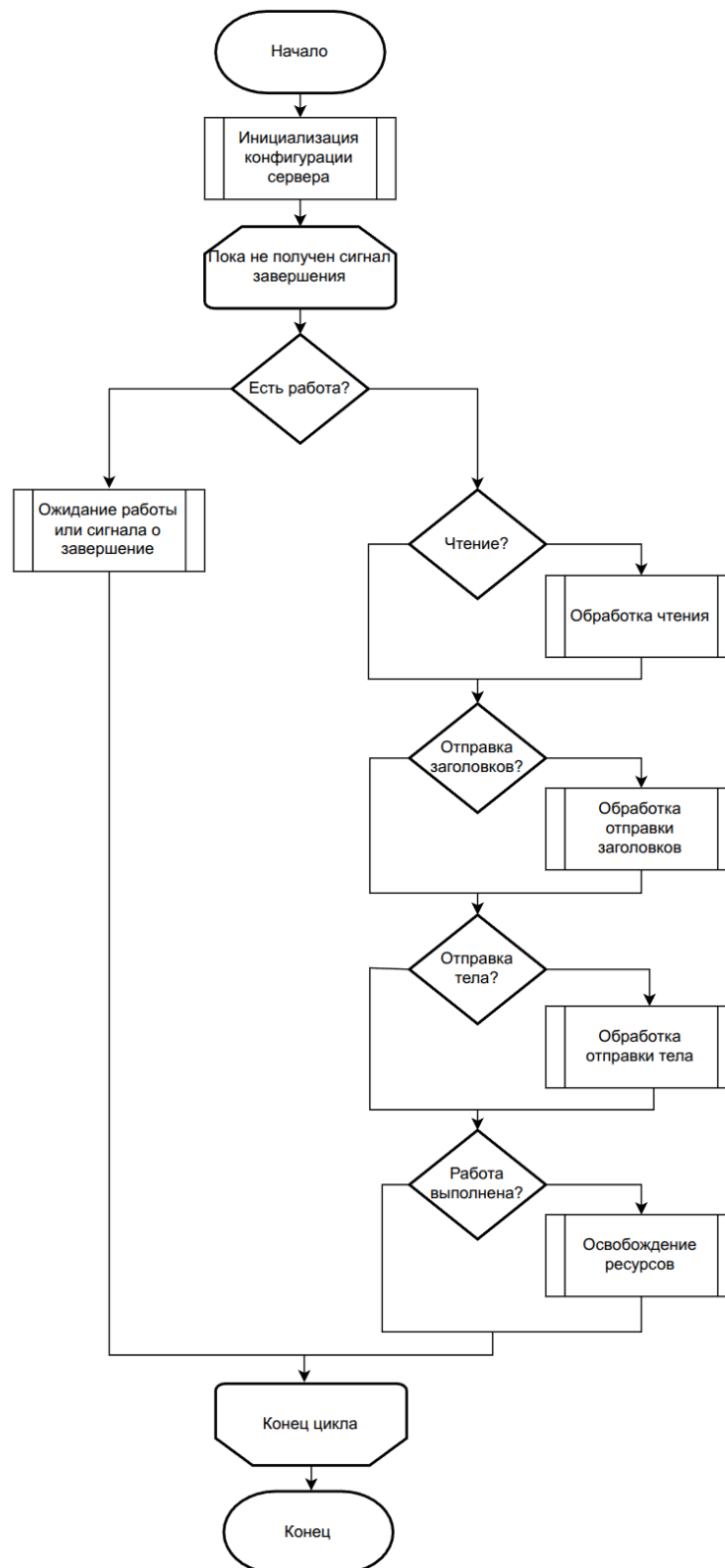


Рисунок 2.4 – Схема алгоритма работы потока обработчика

3 Технологический раздел

В данном разделе описаны средства реализации и программная реализация основных компонентов многопоточного сервера.

3.1 Средства реализации

Для реализации сервера был выбран язык программирования C стандарта C17. Выбор обусловлен следующими причинами:

- прямой доступ к системным вызовам Linux (epoll, sendfile);
- полный контроль над управлением памятью;
- совместимость с POSIX Threads.

Сборка проекта осуществляется с помощью утилиты GNU Make. Компиляция производится компилятором GCC с флагами `-Wall -Wextra -Wpedantic -std=c17`. В режиме отладки используется флаг `-g` и макрос `DEBUG`, в режиме выпуска — флаг оптимизации `-O2`.

3.2 Реализация алгоритма работы потока слушателя

Поток слушатель выполняет приём входящих соединений и регистрацию клиентских событий через подсистему epoll. Каждый поток слушатель создаёт собственный серверный сокет с опцией `SO_REUSEPORT`, что позволяет ядру Linux распределять входящие соединения между несколькими слушателями.

3.2.1 Установка неблокирующего режима

На листинге 3.1 представлена функция перевода файлового дескриптора в неблокирующий режим с помощью системного вызова `fcntl`.

Листинг 3.1 – Установка неблокирующего режима

```
1 int setNonblocking(int fd)
2 {
3     int flags = fcntl(fd, F_GETFL, 0);
4     if (flags == -1)
5         return 10;
6     if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) == -1)
7         return 20;
```

```
8     return 0;
9 }
```

3.2.2 Обработка входящих подключений

Функция `connectionHandle`, представленная на листинге 3.2, вызывает-
ся при готовности серверного сокета.

Листинг 3.2 – Обработка входящих подключений

```
1 void connectionHandle(int serverfd, int epollfd)
2 {
3     while (1)
4     {
5         struct sockaddr_in clientAddr = {0};
6         socklen_t clientLen = sizeof(clientAddr);
7         int clientfd = accept(serverfd,
8                               (struct sockaddr *)&clientAddr, &clientLen);
9
10        if (clientfd < 0)
11        {
12            if (errno == EAGAIN || errno == EWOULDBLOCK)
13                break;
14            break;
15        }
16
17        if (setNonblocking(clientfd) != 0)
18        {
19            close(clientfd);
20            continue;
21        }
22
23        Task_t *task = createTask(clientfd, epollfd);
24        if (!task)
25        {
26            close(clientfd);
27            continue;
28        }
29
30        inet_ntop(AF_INET, &clientAddr.sin_addr,
31                  task->clientip, sizeof(task->clientip));
32
33        struct epoll_event ev = {0};
34        ev.data.ptr = task;
35        ev.events = EPOLLIN | EPOLLET
36                  | EPOLLRDHUP | EPOLLONESHOT;
37
38        if (epoll_ctl(epollfd, EPOLL_CTL_ADD,
```

```

39         clientfd, &ev) == -1)
40     {
41         deleteTask(task);
42         close(clientfd);
43         continue;
44     }
45 }
46 }

```

3.2.3 Реализация алгоритма потока слушателя

На листинге 3.3 представлена функция потока слушателя.

Листинг 3.3 – Реализация потока слушателя

```

1 void *listenerThread(void *args)
2 {
3     ServerConfig_t *conf = (ServerConfig_t *)args;
4
5     int serverfd = socket(AF_INET, SOCK_STREAM, 0);
6     if (serverfd < 0)
7         return NULL;
8
9     int opt = 1;
10    setsockopt(serverfd, SOL_SOCKET, SO_REUSEPORT,
11               &opt, sizeof(opt));
12
13    struct sockaddr_in addr = {0};
14    addr.sin_family = AF_INET;
15    addr.sin_port = htons(conf->port);
16    addr.sin_addr.s_addr = INADDR_ANY;
17
18    bind(serverfd, (struct sockaddr *)&addr, sizeof(addr));
19    listen(serverfd, MAXCON);
20    setNonblocking(serverfd);
21
22    int epollfd = epoll_create1(0);
23
24    struct epoll_event ev = {0};
25    ev.events = EPOLLIN | EPOLLET;
26    ev.data.fd = serverfd;
27    epoll_ctl(epollfd, EPOLL_CTL_ADD, serverfd, &ev);
28
29    /*      ( 5      ) */
30    int timerfd = timerfd_create(CLOCK_MONOTONIC,
31                                TFD_NONBLOCK);
32    struct itimerspec timer_spec = {0};
33    timer_spec.it_value.tv_sec = 5;
34    timer_spec.it_interval.tv_sec = 5;

```

```

35 timerfd_settime(timerfd, 0, &timer_spec, NULL);
36 ev.data.fd = timerfd;
37 ev.events = EPOLLIN;
38 epoll_ctl(epollfd, EPOLL_CTL_ADD, timerfd, &ev);
39
40 /* eventfd */
41 ev.data.fd = conf->shutdownfd;
42 ev.events = EPOLLIN;
43 epoll_ctl(epollfd, EPOLL_CTL_ADD,
44           conf->shutdownfd, &ev);
45
46 struct epoll_event events[MAXEVENTS];
47
48 while (!conf->shutdown)
49 {
50     int nfds = epoll_wait(epollfd, events,
51                           MAXEVENTS, 1000);
52     if (nfds == -1)
53     {
54         if (errno == EINTR) continue;
55         break;
56     }
57
58     for (int i = 0; i < nfds; ++i)
59     {
60         if (events[i].data.fd == conf->shutdownfd)
61             goto cleanup;
62         else if (events[i].data.fd == serverfd)
63             connectionHandle(serverfd, epollfd);
64         else if (events[i].data.fd == timerfd)
65             timerHandle(timerfd);
66         else
67         {
68             Task_t *task =
69                 (Task_t *)events[i].data.ptr;
70             if (events[i].events
71                 & (EPOLLRDHUP | EPOLLHUP))
72             {
73                 epoll_ctl(epollfd, EPOLL_CTL_DEL,
74                           task->clientfd, NULL);
75                 close(task->clientfd);
76                 deleteTask(task);
77                 continue;
78             }
79             push(conf->queue, task);
80         }
81     }
82 }

```

```

83
84 cleanup:
85     close(timerfd);
86     close(epollfd);
87     close(serverfd);
88     return NULL;
89 }

```

3.3 Реализация алгоритма работы потока обработчика

Поток-обработчик извлекает задачи из очереди и обрабатывает HTTP-запросы в соответствии с конечным автоматом клиентского соединения.

Листинг 3.4 – Реализация функции чтения HTTP-запроса

```

1  int readingHandle(Task_t *task)
2  {
3      while (1)
4      {
5          if (task->readPos >= task->readLen - 1)
6              return -1;
7
8          ssize_t n = recv(task->clientfd,
9                          task->readBuf + task->readPos,
10                         task->readLen - task->readPos - 1,
11                         MSG_DONTWAIT);
12
13          if (n > 0)
14          {
15              task->readPos += n;
16              task->readBuf[task->readPos] = '\0';
17              task->lastActivity = time(NULL);
18              if (strstr(task->readBuf, "\r\n\r\n"))
19              {
20                  task->state = SENDING_HEADERS;
21                  return 1;
22              }
23          }
24          else if (n == 0)
25              return -1;
26          else
27          {
28              if (errno == EAGAIN || errno == EWOULDBLOCK)
29                  return 0;
30              return -1;
31          }
32      }
33  }

```

3.3.1 Реализация функции парсинга HTTP-запроса

На листинге 3.5 представлена функция разбора HTTP-запроса.

Листинг 3.5 – Реализация функции парсинг HTTP-запроса

```
1 int parseRequest(Task_t *task)
2 {
3     char *buf = task->readBuf;
4
5     if (strncmp(buf, "GET ", 4) == 0)
6     {
7         task->method = GET;
8         buf += 4;
9     }
10    else if (strncmp(buf, "HEAD ", 5) == 0)
11    {
12        task->method = HEAD;
13        buf += 5;
14    }
15    else
16    {
17        task->method = UNKNOWN;
18        task->statusCode = 405;
19        return 0;
20    }
21
22    char *pathStart = buf;
23    char *pathEnd = strchr(buf, ' ');
24    if (!pathEnd)
25    {
26        task->statusCode = 400;
27        return -1;
28    }
29
30    size_t pathLen = pathEnd - pathStart;
31    task->targetPath = malloc(pathLen + 1);
32    if (!task->targetPath)
33    {
34        task->statusCode = 500;
35        return -1;
36    }
37    memcpy(task->targetPath, pathStart, pathLen);
38    task->targetPath[pathLen] = '\0';
39
40    if (strstr(task->targetPath, ".."))
41    {
42        task->statusCode = 403;
43        return 0;
```

```

44     }
45
46     char *connHeader =
47         strcasestr(task->readBuf, "\r\nConnection:");
48     if (connHeader)
49     {
50         connHeader += 13;
51         while (*connHeader == ' ') connHeader++;
52         if (strncasecmp(connHeader, "close", 5) == 0)
53             task->keepAlive = 0;
54     }
55     return 0;
56 }

```

3.3.2 Реализация функции формирования HTTP-ответа

На листинге 3.6 представлена функция формирования HTTP-ответа.

Листинг 3.6 – Реализация функции формирования HTTP-ответа

```

1  int prepareResponse(Task_t *task)
2  {
3      char filepath[512];
4      struct stat st;
5      const char *mimeType = "text/html";
6      const char *body = NULL;
7      size_t bodyLen = 0;
8
9      if (task->statusCode == 200)
10     {
11         const char *target =
12             (strcmp(task->targetPath, "/") == 0)
13             ? "/index.html" : task->targetPath;
14         snprintf(filepath, sizeof(filepath),
15                 "%s%s", WWW_ROOT, target);
16
17         task->filefd = open(filepath, O_RDONLY);
18         if (task->filefd < 0)
19             task->statusCode = 404;
20         else if (fstat(task->filefd, &st) < 0
21                 || !S_ISREG(st.st_mode))
22         {
23             close(task->filefd);
24             task->filefd = -1;
25             task->statusCode = 404;
26         }
27         else
28         {
29             task->fileSize = st.st_size;

```

```

30         mimeType = getMimeType(filepath);
31     }
32 }
33
34 if (task->statusCode != 200)
35 {
36     if (task->filefd >= 0)
37     {
38         close(task->filefd);
39         task->filefd = -1;
40     }
41     task->fileSize = 0;
42     switch (task->statusCode)
43     {
44         case 403: body = ERROR_403_BODY; break;
45         case 404: body = ERROR_404_BODY; break;
46         case 405: body = ERROR_405_BODY; break;
47         default:  body = ERROR_500_BODY; break;
48     }
49     bodyLen = strlen(body);
50 }
51
52 size_t contentLength =
53     (task->filefd >= 0) ? task->fileSize : bodyLen;
54
55 char headers[1024];
56 int headersLen = snprintf(headers, sizeof(headers),
57     "HTTP/1.1 %d %s\r\n"
58     "Content-Type: %s\r\n"
59     "Content-Length: %zu\r\n"
60     "Connection: %s\r\n\r\n",
61     task->statusCode,
62     getStatusText(task->statusCode),
63     mimeType, contentLength,
64     task->keepAlive ? "keep-alive" : "close");
65
66 size_t totalSize = headersLen
67     + (body ? bodyLen : 0);
68 task->writeBuf = malloc(totalSize);
69 if (!task->writeBuf)
70     return -1;
71
72 memcpy(task->writeBuf, headers, headersLen);
73 if (body)
74     memcpy(task->writeBuf + headersLen,
75         body, bodyLen);
76
77 task->writeLen = totalSize;

```

```

78     task->writePos = 0;
79     return 0;
80 }

```

3.3.3 Реализация функции отправки HTTP-заголовков

На листинге 3.7 представлена функция отправки HTTP-заголовков клиенту.

Листинг 3.7 – Реализация функции отправки HTTP-заголовков

```

1  int sendingHeadersHandle(Task_t *task)
2  {
3      while (task->writePos < task->writeLen)
4      {
5          ssize_t n = send(task->clientfd,
6                          task->writeBuf + task->writePos,
7                          task->writeLen - task->writePos,
8                          MSG_DONTWAIT);
9          if (n > 0)
10             task->writePos += n;
11         else if (n < 0)
12         {
13             if (errno == EAGAIN || errno == EWOULDBLOCK)
14                 return 0;
15             return -1;
16         }
17     }
18
19     if (task->filefd >= 0 && task->method == GET)
20         task->state = SENDING_BODY;
21     else
22         task->state = DONE;
23     return 1;
24 }

```

3.3.4 Реализация функции отправки тела ответа

На листинге 3.8 представлена функция отправки тела ответа.

Листинг 3.8 – Реализация функции отправки тела ответа через sendfile

```

1  int sendingBodyHandle(Task_t *task)
2  {
3      while (task->filePos < task->fileSize)
4      {
5          off_t offset = task->filePos;
6          ssize_t sent = sendfile(task->clientfd,
7                                 task->filefd, &offset,

```

```

8         task->fileSize - task->filePos);
9
10        if (sent > 0)
11            task->filePos += sent;
12        else if (sent < 0)
13        {
14            if (errno == EAGAIN
15                || errno == EWOULDBLOCK)
16                return 0;
17            return -1;
18        }
19    }
20    task->state = DONE;
21    return 1;
22 }

```

3.3.5 Реализация функции перерегистрации дескриптора в `epoll`

На листинге 3.9 представлена функция перерегистрации дескриптора в `epoll` после обработки.

Листинг 3.9 – Реализация функции перерегистрации дескриптора в `epoll`

```

1 void rearmEpoll(Task_t *task, uint32_t events)
2 {
3     struct epoll_event ev = {0};
4     ev.data.ptr = task;
5     ev.events = events | EPOLLET
6                 | EPOLLRDHUP | EPOLLONESHOT;
7     epoll_ctl(task->epollfd, EPOLL_CTL_MOD,
8                 task->clientfd, &ev);
9 }

```

3.3.6 Реализация функции потока обработчика

На листинге 3.10 представлена функция потока обработчика. Поток в цикле извлекает задачи из очереди и обрабатывает их в соответствии с текущим состоянием конечного автомата. Переходы между состояниями: `READING` → `SENDING_HEADERS` → `SENDING_BODY` → `DONE`. При поддержке `keep-alive` после состояния `DONE` соединение возвращается в состояние `READING` для обработки следующего запроса. При ошибке `EAGAIN` дескриптор перерегистрируется в `epoll`, и задача возвращается в пул событий.

Листинг 3.10 – Реализация функции потока обработчика

```

1 void *workerThread(void *args)
2 {
3     ServerConfig_t *conf = (ServerConfig_t *)args;
4     int needClose;
5
6     while (!conf->shutdown)
7     {
8         Task_t *task = NULL;
9         needClose = 0;
10
11         if (pop(conf->queue, &task) != 0)
12             break;
13         if (!task)
14             continue;
15
16         atomic_fetch_add(&total_requests, 1);
17
18         switch (task->state)
19         {
20             case READING:
21             {
22                 int rc = readingHandle(task);
23                 if (rc == 1)
24                 {
25                     if (parseRequest(task) < 0
26                         || prepareResponse(task) < 0)
27                     {
28                         needClose = 1; break;
29                     }
30                     rc = sendingHeadersHandle(task);
31                     if (rc == 0)
32                     {
33                         rearmEpoll(task, EPOLLOUT);
34                         continue;
35                     }
36                     else if (rc < 0)
37                     {
38                         needClose = 1; break;
39                     }
40                     if (task->state == SENDING_BODY)
41                     {
42                         rc = sendingBodyHandle(task);
43                         if (rc == 0)
44                         {
45                             rearmEpoll(task, EPOLLOUT);
46                             continue;
47                         }
48                         else if (rc < 0)

```

```

49         {
50             needClose = 1; break;
51         }
52     }
53 }
54 else if (rc == 0)
55 {
56     rearmEpoll(task, EPOLLIN);
57     continue;
58 }
59 else
60 {
61     needClose = 1; break;
62 }
63 }
64 /* fallthrough */
65 case SENDING_HEADERS:
66 {
67     int rc = sendingHeadersHandle(task);
68     if (rc == 0)
69     {
70         rearmEpoll(task, EPOLLOUT);
71         continue;
72     }
73     else if (rc < 0)
74     {
75         needClose = 1; break;
76     }
77     if (task->state != SENDING_BODY)
78         break;
79 }
80 /* fallthrough */
81 case SENDING_BODY:
82 {
83     int rc = sendingBodyHandle(task);
84     if (rc == 0)
85     {
86         rearmEpoll(task, EPOLLOUT);
87         continue;
88     }
89     else if (rc < 0)
90     {
91         needClose = 1; break;
92     }
93 }
94 /* fallthrough */
95 case DONE:
96     if (task->keepAlive)

```

```

97         {
98             close(task->filefd);
99             task->filefd = -1;
100             free(task->writeBuf);
101             task->writeBuf = NULL;
102             free(task->targetPath);
103             task->targetPath = NULL;
104             task->state = READING;
105             task->readPos = 0;
106             task->writeLen = 0;
107             task->writePos = 0;
108             task->fileSize = 0;
109             task->filePos = 0;
110             task->statusCode = 200;
111             task->method = UNKNOWN;
112             rearmEpoll(task, EPOLLIN);
113             continue;
114         }
115         needClose = 1;
116         break;
117     }
118
119     if (needClose)
120     {
121         epoll_ctl(task->epollfd, EPOLL_CTL_DEL,
122                 task->clientfd, NULL);
123         close(task->clientfd);
124         deleteTask(task);
125     }
126 }
127 return NULL;
128 }

```

3.4 Реализация алгоритма работы главного потока сервера

Главный поток сервера выполняет инициализацию всех подсистем и ожидает сигнал завершения. На листинге 3.11 представлена функция main

Листинг 3.11 – Реализация функции главного потока сервера

```

1 int main(int argc, char **argv)
2 {
3     ServerConfig_t conf = {0};
4     int rc = parseArgs(argc, argv, &conf);
5     if (rc != 0)
6         return (rc == 1) ? EXIT_SUCCESS : EXIT_FAILURE;
7
8     signal(SIGPIPE, SIG_IGN);

```

```

9      init_logger("logs/server.log");
10
11      sigset_t sigset;
12      sigemptyset(&sigset);
13      sigaddset(&sigset, SIGINT);
14      sigaddset(&sigset, SIGTERM);
15      pthread_sigmask(SIG_BLOCK, &sigset, NULL);
16
17      conf.queue = createQueue();
18      conf.shutdown = 0;
19      conf.shutdownfd = eventfd(0, EFD_NONBLOCK);
20
21      pthread_t *workers =
22          calloc(conf.workers, sizeof(pthread_t));
23      pthread_t *listeners =
24          calloc(conf.listeners, sizeof(pthread_t));
25
26      /* */
27      for (int i = 0; i < conf.workers; ++i)
28          pthread_create(&workers[i], NULL,
29                        &workerThread, &conf);
30      for (int i = 0; i < conf.listeners; ++i)
31          pthread_create(&listeners[i], NULL,
32                        &listenerThread, &conf);
33
34      /* */
35      int sig;
36      sigwait(&sigset, &sig);
37
38      /* */
39      conf.shutdown = 1;
40      shutdownQueue(conf.queue);
41
42      uint64_t val = 1;
43      write(conf.shutdownfd, &val, sizeof(val));
44
45      for (int i = 0; i < conf.listeners; ++i)
46          pthread_join(listeners[i], NULL);
47      for (int i = 0; i < conf.workers; ++i)
48          pthread_join(workers[i], NULL);
49
50      free(workers);
51      free(listeners);
52      close(conf.shutdownfd);
53      deleteQueue(conf.queue);
54      close_logger();
55      return EXIT_SUCCESS;
56  }

```

4 Исследовательский раздел

В данном разделе описана методика нагрузочного тестирования реализованного многопоточного HTTP-сервера, приведены результаты экспериментов и сформулированы выводы о выборе оптимальной конфигурации.

4.1 Условия проведения исследования

Нагрузочное тестирование проводилось с помощью утилиты `wrk`, которая генерирует HTTP-запросы в многопоточном режиме.

Тестирование проводилось на машине со следующими характеристиками:

- операционная система Ubuntu 22.04.5 LTS;
- объем оперативной памяти - 56 ГБ;
- объем свободного пространства NVME2 накопителя Samsung 980 EVO — 500 ГБ [9];
- процессор Intel Core i9-11900KF @ 3.50 GHz

Параметры исследования:

- количество потоков `wrk`: 8;
- длительность каждого прогона: 3 секунды;
- количество прогонов на каждую комбинацию параметров: 10;
- запрашиваемый ресурс: статическая HTML-страница;
- уровни одновременных подключений: 10 000, 20 000, 40 000, 80 000, 100 000.

Тестировались шесть конфигураций сервера, различающихся количеством потоков-слушателей (L) и потоков-обработчиков (W): ($L=2, W=2$), ($L=2, W=4$), ($L=2, W=8$), ($L=3, W=4$), ($L=3, W=8$), ($L=4, W=4$).

Для каждой комбинации параметров вычислялись средние значения трёх метрик: количество запросов в секунду (req/s), средняя задержка ответа (мс) и скорость передачи данных (МБ/с).

4.2 Результаты тестирования

В таблице 4.1 приведены усреднённые результаты нагрузочного тестирования по всем конфигурациям и уровням нагрузки.

Таблица 4.1 – Результаты нагрузочного тестирования (средние значения)

<i>L</i>	<i>W</i>	Подключения	Запросов/с	Задержка, мс	Передача, МБ/с
2	2	10 000	95 247	99,15	868,9
2	2	20 000	48 090	122,82	438,4
2	2	40 000	2 569	30,80	23,5
2	2	80 000	927	31,10	8,5
2	2	100 000	709	30,80	6,5
2	4	10 000	150 154	63,12	1 369,1
2	4	20 000	65 770	86,19	599,7
2	4	40 000	3 450	24,49	31,5
2	4	80 000	1 240	24,30	11,3
2	4	100 000	924	24,85	8,4
2	8	10 000	168 414	56,79	1 535,0
2	8	20 000	71 480	68,83	651,8
2	8	40 000	3 726	23,45	34,0
2	8	80 000	1 314	23,80	12,0
2	8	100 000	1 001	23,70	9,1
3	4	20 000	68 303	75,80	622,8
3	4	40 000	3 535	23,90	32,3
3	4	80 000	1 185	25,25	10,8
3	4	100 000	922	23,90	8,4
3	8	20 000	68 446	61,29	624,1
3	8	40 000	3 604	23,85	32,9
3	8	80 000	1 251	23,74	11,4
3	8	100 000	982	24,28	9,0
4	4	10 000	110 845	103,18	1 008,5
4	4	20 000	24 028	212,69	219,1
4	4	40 000	2 378	35,98	21,7
4	4	80 000	970	34,26	8,9

4.3 Исследование влияния количества потоков на пропускную способность

На рисунке 4.1 представлена зависимость количества обработанных запросов в секунду от числа слушателей и обработчиков для трёх уровней нагрузки.

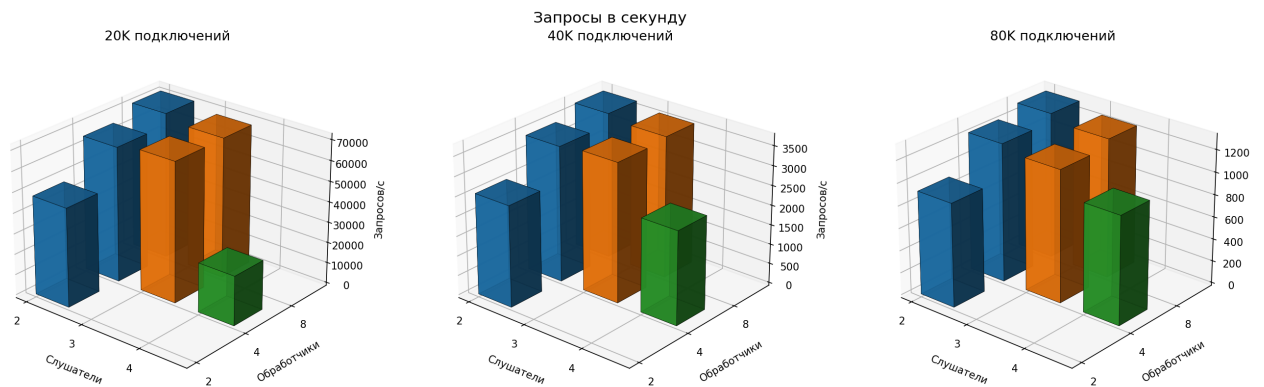


Рисунок 4.1 – Зависимость пропускной способности от конфигурации потоков

4.4 Исследование влияния количества потоков на задержку

На рисунке 4.2 представлена зависимость средней задержки ответа от конфигурации потоков.

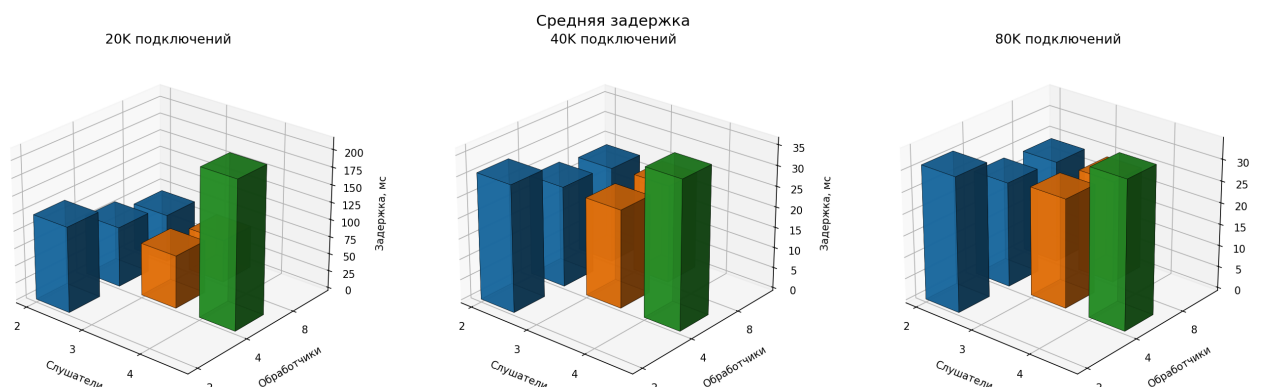


Рисунок 4.2 – Зависимость средней задержки от конфигурации потоков

4.5 Исследование влияния количества потоков на скорость передачи

На рисунке 4.3 представлена зависимость скорости передачи данных от конфигурации потоков.

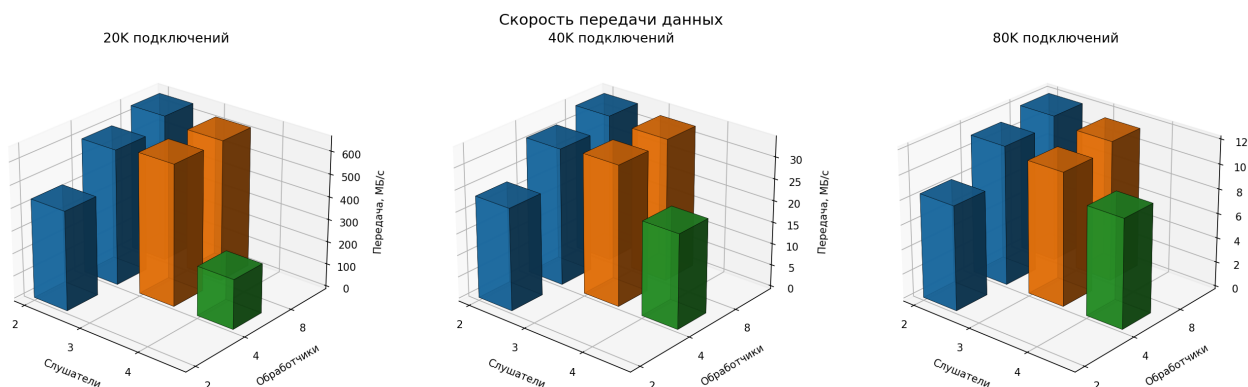


Рисунок 4.3 – Зависимость скорости передачи данных от конфигурации ПОТОКОВ

4.6 Исследование конфигурации $L=2, W=8$

На рисунке 4.4 представлена трёхмерная зависимость задержки и скорости передачи от числа подключений для конфигурации ($L=2, W=8$), аппроксимированная полиномом третьей степени.

$L=2, W=8$: Тепловая карта производительности

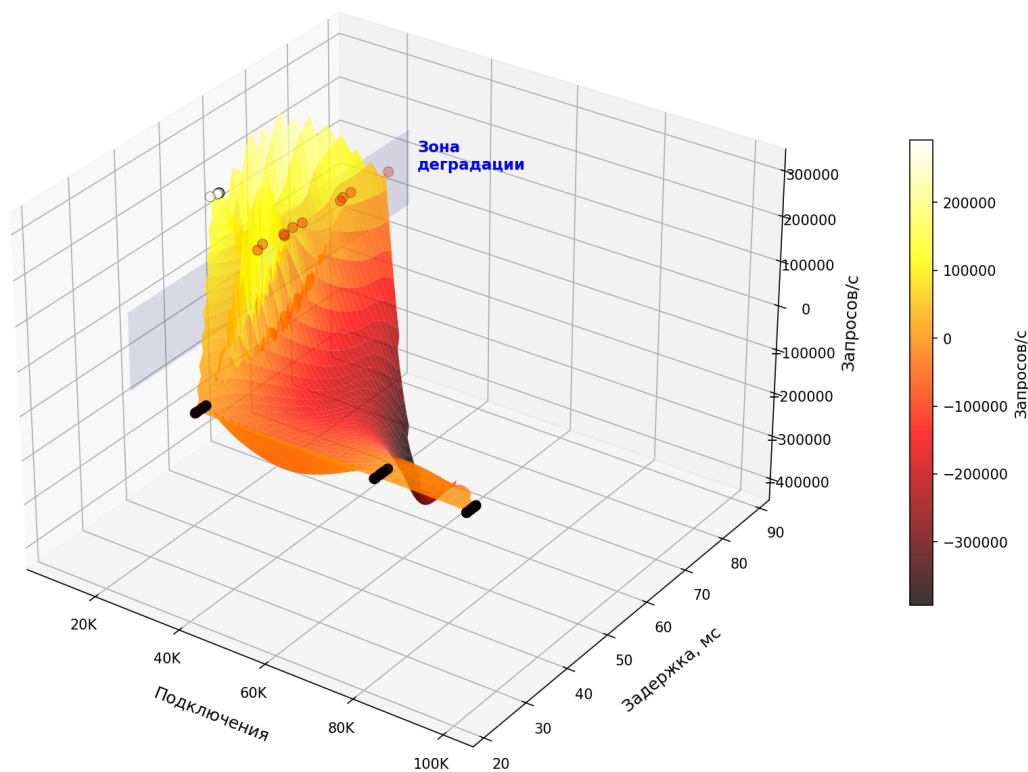


Рисунок 4.4 – Конфигурация $L=2, W=8$: зависимость задержки и скорости передачи от числа подключений

4.7 Выводы

По результатам нагрузочного тестирования установлено следующее.

- 1) Увеличение количества потоков-обработчиков оказывает наибольшее влияние на производительность сервера. При фиксированном числе слушателей $L=2$ увеличение числа обработчиков с 2 до 8 повышает пропускную способность на 77% (с 95 247 до 168 414 req/s при 10 000 подключений) и снижает задержку на 43% (с 99,15 до 56,79 мс).
- 2) Увеличение количества потоков-слушателей свыше двух не приводит к росту производительности. Конфигурация ($L=4$, $W=4$) показала результаты в 2-3 раза хуже, чем ($L=2$, $W=4$), что объясняется избыточной конкуренцией слушателей за входящие соединения при использовании `SO_REUSEPORT`.
- 3) При нагрузках свыше 40 000+ подключений различия между конфигурациями нивелируются: узким местом становятся системные ограничения ОС, а не архитектура пула потоков.
- 4) Оптимальной конфигурацией сервера является ($L=2$, $W=8$). Данная конфигурация обеспечивает наибольшую пропускную способность и наименьшую задержку на всех уровнях нагрузки.

ЗАКЛЮЧЕНИЕ

Цель курсовой работы достигнута: был разработан статический веб-сервер. Для достижения поставленной цели были выполнены следующие задачи:

- проанализирована архитектура компьютерной сети, протоколы и системные вызовы;
- описаны основные компоненты и алгоритмы работы веб-сервера.
- реализован сервер;
- проведено нагрузочное тестирование разработанного сервера;

На основе исследования была определена оптимальная конфигурация разработанного сервера.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Таненбаум Э.* Компьютерные сети. — 5-е изд. — СПб. : Питер, 2018. — 592 с. — (дата обращения: 5.2.2026).
2. *Куроуз Д. Ф., Росс К. В.* Компьютерные сети. — 2-е изд. — М. : ЗАО Издательский дом «Питер», 2004. — 432 с. — (дата обращения: 5.2.2026).
3. Мануал Linux — pthreads(7). — URL: <https://man7.org/linux/man-pages/man7/pthreads.7.html> (дата обращения: 5.2.2026).
4. The Native POSIX Thread Library. — URL: <https://lwn.net/Articles/10710/> (дата обращения: 5.2.2026).
5. The Native POSIX Thread Library for Linux. — URL: <https://www.akkadia.org/drepper/nptl-design.pdf> (дата обращения: 5.2.2026).
6. *Керриски М.* Linux API. — М. : Питер, 2018. — 400 с. — (дата обращения: 5.2.2026).

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе

Презентация содержит 10 слайдов.